

Exame de Programação Funcional

10 de Julho de 2010

DCC/FCUP

2ª Parte

Nome: _____

Nº mecanográfico: _____

- Esta 2ª parte contém 5 questões e 3 páginas.
- Responda às questões 1 e 2 no espaço marcado no enunciado.
- Responda às questões 3, 4 e 5 numa folha de exame separada.

1. Escreva uma definição da função *binario* $:: Int \rightarrow [Int]$ que calcula a representação de um inteiro positivo em algarismos binários (0 ou 1). A lista resultado deve estar ordenada do algarismo menos significativo para o mais significativo. Exemplo: *binario* 6 = [0, 1, 1].

2. Escreva uma função $partir :: Int \rightarrow [a] \rightarrow [[a]]$ tal que ‘ $partir\ n$ ’ decompõe uma lista em sub-listas cuja concatenação dá a lista original e tal que cada sub-lista tem comprimento n (excepto, possivelmente, a última). Exemplo: $partir\ 5\ \text{“abcdefghijkl”} = [\text{“abcde”}, \text{“fghij”}, \text{“kl”}]$.

Responda às questões 3, 4 e 5 numa folha de exame separada.

3. Considere a representação de uma relação binária nos inteiros como uma lista de pares.

$$\text{type } Rel = [(Int, Int)]$$

Escreva uma função *transitiva* $:: Rel \rightarrow Bool$ que verifique se uma relação é transitiva (R é transitiva se e só se $(x, y) \in R \wedge (y, z) \in R \implies (x, z) \in R$ para todos x, y, z). Exemplos:

$$\text{transitiva } [(1, 3), (1, 2), (2, 3)] = True$$

$$\text{transitiva } [(1, 2), (2, 3)] = False$$

4. Considere a definição do tipo de árvores de pesquisa apresentada nas aulas.

$$\text{data } Arv\ a = \text{Vazia} \mid \text{No } a\ (Arv\ a)\ (Arv\ a)$$

- (a) Escreva uma definição recursiva da função *listar* $:: Arv\ a \rightarrow [a]$ que lista todos os valores numa árvore por ordem infixada da esquerda para a direita.
- (b) Escreva uma definição da função *reflect* $:: Arv\ a \rightarrow Arv\ a$ que recursivamente troca os lados esquerdos e direitos de uma árvore. Exemplo:

$$\text{reflect} \left(\begin{array}{c} & 2 & \\ \swarrow & & \searrow \\ 1 & & 3 \\ \swarrow & & \searrow \\ & 4 & 5 \end{array} \right) = \begin{array}{c} & 2 & \\ \swarrow & & \searrow \\ & 3 & \\ \swarrow & & \searrow \\ 5 & & 4 \end{array}$$

- (c) Usando indução sobre árvores, prove que

$$\text{listar } (\text{reflect } t) = \text{reverse } (\text{listar } t)$$

em *reverse* $:: [a] \rightarrow [a]$ é a função do prelúdio-padrão que inverte uma lista.

5. Considere a seguinte definição da função *inserir* $:: Int \rightarrow [Int] \rightarrow [Int]$ que insere um valor numa lista crescente de inteiros mantendo a ordenação.

$$\text{inserir } x\ [] = [x]$$

$$\text{inserir } x\ (y : ys) \mid x \leq y = x : y : ys$$

$$\text{inserir } x\ (y : ys) \mid x > y = y : \text{inserir } x\ ys$$

Usando indução sobre listas, prove que $\text{length } (\text{insert } x\ xs) = 1 + \text{length } xs$.